

Financial Analyzer

Programátorská dokumentace

Úvod

Právě čtete programátorskou (vývojářskou) dokumentaci k programu Financial Analyzer vypracovaného jakožto zápočtový program předmětu Programování v C++ studentem Markem Pučejdlem ve školním roce 2025/2026.

Dokumentace slouží k high-level pochopení fungování aplikace, diskutuje interakce modulů i jejich vnitřní fungování. Zároveň pak uvádí a zdůvodňuje odchylky od zadání a poučení vývojem získaná.

Zápočtovým programem konzolová aplikace je *Kvantitativní analyzátor finančního portfolia*, nazvaný Financial Analyzer. Uživateli umožňuje zadat a poté sledovat své portfolio aktiv a archivovat změny v něm. Zároveň pomocí kvantitativních metod na příkaz vypočte statistiky minulého vývoje portfolia a pomocí základních metod odhadne budoucí vývoj. K zisku dat o finančních instrumentech využívá veřejná API. Aplikace podporuje akcie, hlavní zahraniční měny a kryptoměny.

Top-level architektura

Projekt sestává z pouze inicializačního main.cpp a 6 dalších modulů, konkrétně:

- a) APIMiddleman.cpp/.hpp - umožňuje abstrakci nad přístupem k finančním API
- b) financialCalc.cpp/.hpp - poskytuje finanční výpočty
- c) runtimePortfolio.cpp/.hpp - implementuje běhovou datovou třídu RTPortfolio
- d) snapshoter.cpp/.hpp - umožňuje uložit a načíst RTPortfolio ze souboru
- e) CLIPrinter.cpp/.hpp - obsluhuje výpis dat do konzole
- f) userInterface.cpp/.hpp - interpretuje příkazy uživatele a obsluhuje ostatní moduly

Zobrazení vzájemného vztahu schématicky:

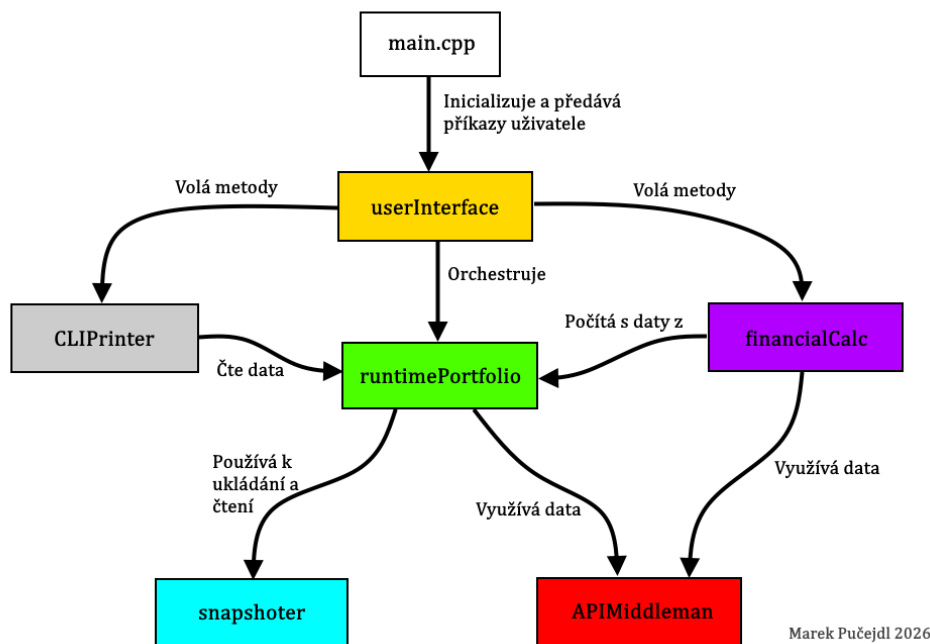


Schéma naznačuje, jaké moduly daný modul využívá (includeuje). Postup vývoje by tak přibližně šel znázornit DAGem s obrácenými šipkami.

Program také využívá dvě externí knihovny.

cpr na síťovou komunikaci. Ten je použit výlučně v APIMiddleman, kde zasílá požadavky na API.

nlohmann/json pro parsování json formátu je pak použita jednak v APIMiddleman pro parsování příchozích dat z API, ale také v snapshoter, kde se používá pro převod do/z uložitelného json textového formátu.

Vnitřní rozbor modulů

a) *APIMiddleman*

Modul *APIMiddleman* slouží jako komunikační vrstva mezi aplikací a externími finančními webovými službami. Abstrahuje nad veřejnými API a ulehčuje tak používání těchto datových zdrojů v jiných modulech.

Vzhledem k tomu, že projekt spoléhá na bezplatné verze API, které mají často striktní limity počtu dotazů nebo omezenou nabídku dat, integruje modul více nezávislých zdrojů. Potřebné autentizační klíče pro tyto služby se načítají dynamicky a bezpečně ze souboru *config.json* pomocí interní struktury *ApiKeys*.

Na nejnižší úrovni jsou implementovány specifické třídy odpovídající konkrétním API, jako je *FinnHubChannel* pro aktuální ceny akcií, *TwelveDataChannel* pro jejich historická data, *FrankfurterChannel* pro měnové kurzy, *BinanceChannel* pro kryptoměny a *FredChannel* pro úrokové sazby centrálních bank.

Modul pak tyto kanály abstrahuje ještě výš bez fixace na zdroj, konkrétně do 3 tříd: *StockDataChannel*, *CashDataChannel* a *CryptoDataChannel*. Zároveň tyto třídy veškeré požadavky zapouzdřují v *try-catch* blocích. Pokud dojde k chybě, vypršení limitu dotazů nebo k absenci dat, abstrakce zachytí výjimku a vrátí *std::optional*s hodnotou *std::nullopt*. Dotazy na aktuální data tak nezpůsobují výjimky.

b) *financialCalc*

Modul *financialCalc* v sobě implementuje výpočetní a analytické metody. Zastřešuje jej třída *FinancialCalculator*, která provádí matematické operace nad daty uživatelského portfolia.

Mezi její základní funkce patří agregace aktuální celkové hodnoty držených aktiv a výpočet jejich dosavadní výkonnosti porovnáním nákupních a tržních cen. Modul dále dokáže pomocí základních metod odhadnout budoucí vývoj pomocí funkce *fixedYield*, která modeluje zhodnocení na základě složeného úročení. Dále obsahuje funkci pro backtesting, která vypočte statistiky minulého vývoje portfolia tak, že porovná aktuální stav s historickými cenami k uživatelem zadanému datu.

Kromě těchto metrik modul implementuje pokročilejší kvantitativní metody a nástroje technické analýzy. Nabízí výpočet indikátoru *RSI* a vyhodnocení stavu trhu pomocí *Bollingerových pásem*. Dále také obsahuje pravděpodobnostní simulaci *Monte Carlo*, pomocí které umí odhadnout šanci na dosažení určité ceny nějakého instrumentu v daném časovém horizontu.

c) runtimePortfolio

Modul *runtimePortfolio* slouží jako centrální běhové úložiště dat aplikace. Jeho hlavní součástí je třída *RTPortfolio*, která v paměti udržuje aktuální stav uživatelského portfolia rozděleného do tří hlavních kategorií: akcie, hotovost a kryptoměny. Jednotlivá aktiva jsou reprezentována strukturou *instrumentPosition*, jež kromě identifikátoru (tickeru) a drženého množství uchovává i důležité stavové údaje, jako je průměrná nákupní cena, aktuální tržní cena a dividendový či úrokový výnos. Kromě samotných pozic třída striktně eviduje i celkovou bilanci vložených a vybraných prostředků v amerických dolarech.

Z hlediska chování implementuje tento modul veškerou transakční logiku. Vklady a výběry hotovosti, nákupy a částečné či úplné prodeje konkrétních instrumentů. Při transakcích a při explicitních požadavcích na aktualizaci dat (*loadActivePrices*, *loadActiveYields*) se modul přímo dotazuje vrstvy *APIMiddleman*. Právě na této úrovni probíhá vyhodnocení chyb ze sítě: pokud API vrátí prázdnou hodnotu *std::nullopt* je operace nákupu nebo prodeje bezpečně zrušena, do konzole se vypíše varování a stav portfolia zůstane nedotčen. Pro zachování portfolia mezi jednotlivými spuštěními programu pak třída poskytuje metody pro uložení a načtení, které přímo využívají služeb modulu *snapshoter*.

d) snapshoter

Modul *snapshoter* (reprezentovaný třídou *Snapshoter*) má na starosti perzistenci dat, tedy ukládání a načítání stavu portfolia na lokální disk. K převodu dat z paměti aplikace do textového formátu využívá externí knihovnu *nlohmann/json*. Ta umožňuje snadnou serializaci celého objektu *RTPortfolio*, včetně seznamů všech držených akcií, kryptoměn, hotovosti a agregovaných dat o historických vkladech či výběrech, do přehledné struktury formátu JSON. Zpětně pak modul umí tento JSON soubor načíst, rozparsovat a plně zrekonstruovat běhový stav aplikace do nového objektu portfolia.

Na disku se snímky portfolia ukládají do dedikované složky (definované pomocí *storagePath*), kterou si modul v případě její absence umí sám automaticky vytvořit. Jednotlivé soubory jsou pojmenovány jednoduše podle aktuálního data vytvoření (ve formátu YYYY-MM-DD), což aplikaci velmi usnadňuje jejich pozdější dohledávání, ať už pro standardní načtení stavu, nebo pro historický backtesting. Případné chyby jsou opět propagovány pomocí *std::optional*.

e) **CLIPrinter**

Modul *CLIPrinter* je zodpovědný za formátovaný výpis dat do standardního výstupu. Funguje jako čistě prezentační vrstva, která přijímá již zpracovaná data z ostatních modulů – ať už jde o aktuální stav objektu *RTPortfolio* nebo o vypočtené struktury *perfRatios* z finanční kalkulačky a pomocí formátovacích nástrojů standardní knihovny je převádí do přehledné a uživatelsky čitelné podoby. Třída poskytuje sadu specializovaných metod pro různé kontexty, od výpisu základních pozic a celkových agregovaných statistik, přes tabulkové reporty výkonnosti a backtestingu, až po textovou interpretaci pokročilých indikátorů, jako jsou RSI, Bollingerova pásma či výsledky Monte Carlo simulace. Vyčlenění této logiky do samostatného modulu výrazně zpřehledňuje kód a odděluje vizuální reprezentaci dat od jejich zpracování a řízení aplikace.

f) **userInterface**

Modul *userInterface* tvoří hlavní řídicí centrum aplikace a funguje jako orchestrátor propojující všechny ostatní části systému. Jeho stěžejní komponentou je třída *CLIManager*, která si udržuje hlavní běhovou instanci uživatelského portfolia (*RTPortfolio*).

Modul pak udržuje běhovou komunikační smyčku s uživatelem a interpretuje jeho příkazy, které dostává jakožto vstupní řádky on *main.cpp*. Provádění případů je pak vykonáno skrze delegaci na ostatní moduly a případné provázání jejich funkcionalit. Následně pomocí některé z metod *CLIPrinter* uživateli odpoví.

Rozšiřitelnost (pro případné navázání)

Díky modulárnímu návrhu a striktnímu oddělení odpovědností je aplikace jednoduše rozšiřitelná o nové třídy aktiv (dluhopisy, deriváty), nové analytické výpočty či o nová API. Kvůli vyčlenění interakce s uživatelem do *userInterface* a *CLIPrinter* by pak aplikaci bylo možné dodělat GUI rozhraní přidáním odpovídajících modulů.

Dalším námětem na rozšíření by pak mohlo být nahrazení textového JSON modulu *snapshoter* nějakým pokročilejším databázovým řešením. To by umožnilo složitější dotazy nad interními daty, jakož i snížilo nároky na čas i paměť.

V souvislosti s tím by pak mohlo být uživateli umožněno mít více aktivních portfolií najednou, kde by bylo nutné akorát ošetřit názvy snapshotů.

Odchytky od zadání a jejich zdůvodnění

Domnívám se, že program odpovídá detailní specifikaci dojednané v zadání. Přesto jsem však byl donucen některé části pozměnit. Níže reflektuji, proč se tak stalo.

Program oproti zadání nakonec neobsahuje Discounted Cash Flow model pro predikci vývoje cen akcií a Metcalfův zákon pro predikci pohybů kryptoměn. Obě tyto služby narazili na zásadní problém, a to nedostatek vhodných API. Tyto výpočty vyžadují *fundamentální data*, která poskytovatelé dat typicky dávají za paywall. Při snaze dodržet free režim aplikace tak nebylo možné tyto výpočty provést.

Program také neobsahuje podporu dluhopisů. Zde narážíme nejen na problém s daty, která by opět bylo až nemožné zdarma získat, ale také odlišné chování tohoto instrumentu (vypršení, rolování, face value/úrok...).

Aplikace pak poskytuje uživateli čistě CLI. GUI nebylo implementováno, protože by vyžadovalo další zásadní rozšíření. Vzhledem k tomu, jakou výzvou se ukázaly být ostatní části jsem tedy tuto možnost vylepšení (nabízenou v detailní specifikaci), nevyužil.